Robot Wall Following by Reinforcement Learning

Khushei Meghana Meda
COMPSCI-603 Robotics, Spring 2026
University of Massachusetts Amherst
kmeda@umass.edu

***Abstract*— This paper presents a reinforcement learning approach to the robot wall-following problem using a simulated Triton robot equipped with a 2D LiDAR sensor in the Gazebo simulator. Two temporal-difference learning algorithms are implemented and compared: Q-learning (off-policy) and SARSA (on-policy). The environment is represented as a compact discrete state space of 27 states, constructed by dividing the LiDAR field of view into three angular sectors—front, front-right, and right—and discretizing each sector distance into three bins: close, desired, and far. The robot selects from five discrete velocity commands ranging from hard-left turns to hard-right turns. A shaped reward function encourages maintaining a desired wall distance of approximately 0.75 m on the right side, penalizes collisions and excessive spinning, and rewards forward progress. Q-values are initialized to zero, and both algorithms learn entirely from scratch through interaction with the environment. Both algorithms employ ϵ-greedy exploration with multiplicative decay. Training is conducted over 300–400 episodes with the robot spawned at varied positions including straight walls, corners, a thin freestanding I-beam obstacle, and open space. Experimental results demonstrate that both algorithms learn policies enabling the robot to follow walls in all five required scenarios. SARSA converges significantly faster, reaching near-zero accumulated reward within 50–75 episodes compared to 250–300 for Q-learning. Qualitative evaluation reveals that SARSA produces smoother wall-following trajectories with less oscillation along straight walls, consistent with its on-policy nature providing more accurate Q-value estimates at state boundaries.**

I. INTRODUCTION

Wall following is a fundamental capability for autonomous mobile robots, enabling navigation through unknown environments by maintaining a fixed distance from nearby walls. This strategy allows a robot to traverse the perimeter of an environment, handle corners and obstacles, and reach areas that might otherwise require a complete map of the space. Several methods have been used to address this problem, including control-rule based methods and genetic programming [3].

In this project, we address the wall-following problem using reinforcement learning (RL). Rather than programming explicit control rules for every possible situation, we allow the robot to learn an effective policy through interaction with its environment. The robot receives observations from a 2D LiDAR sensor, selects discrete motor commands, and receives scalar reward signals encoding the quality of its behavior. Over hundreds of training episodes, the robot discovers a policy that maintains a desired distance $d_w \approx 0.75\,\text{m}$ from walls on its right side while moving forward.

We implement and compare two classic temporal-difference RL algorithms: Q-learning [1] and SARSA [2]. Q-learning is off-policy, updating Q-values toward the maximum possible future reward. SARSA is on-policy, updating toward the reward of the action actually taken under the current policy. This distinction has practical implications for the quality and stability of the learned wall-following behavior.

The robot used is the Triton robot simulated in Gazebo, controlled via ROS 1 Noetic. The simulation environment consists of a maze with straight walls, 90-degree corners, and a thin freestanding I-beam obstacle requiring the robot to follow both faces and navigate around the tips.

The remainder of this paper is organized as follows. Section II describes our state representation, action space, reward function, Q-table initialization, and algorithm implementations. Section III presents quantitative and qualitative results. Section IV discusses findings and future work.

II. APPROACH

A. Sensing and Field of View Division

The Triton robot is equipped with a 2D LiDAR providing 360-degree range measurements. For right-wall following, we divide the relevant portion of the field of view into three angular sectors as illustrated in Fig. 1:

- **Front** ($-20°$ to $+20°$): A forward-facing cone for detecting obstacles directly ahead.
- **Front-right** ($-60°$ to $-20°$): An intermediate sector at approximately the 2 o'clock position for detecting upcoming walls or corners.
- **Right** ($-110°$ to $-60°$): The primary wall-tracking sector at approximately the 3 o'clock position for maintaining the desired wall distance.

For each sector, we extract the minimum valid range reading, filtering values below 0.12 m (sensor noise) and above 3.5 m (maximum reliable range).

B. State Representation

Each sector distance is discretized into three bins:

- **Close** ($< 0.40\,\text{m}$): Too near the wall, collision risk.
- **Desired** (0.40–0.90 m): Target distance band around $d_w \approx 0.75\,\text{m}$.
- **Far** ($\geq 0.90\,\text{m}$): Wall lost or open space.

The state is encoded as a single integer $s = 9f + 3r_f + r$, where f, r_f, and r are the bin indices (0, 1, or 2) for the

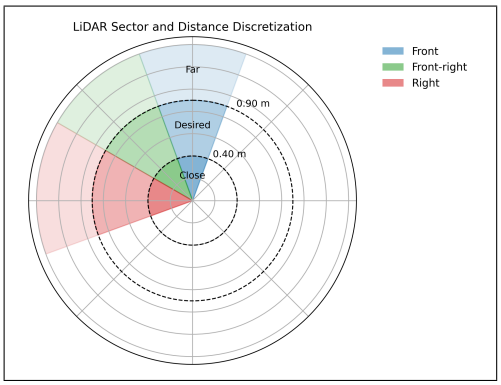

Fig. 1. Division of the LiDAR field of view into three sectors for right-wall following. Angles are measured from the robot's forward axis, with negative angles to the right.

front, front-right, and right sectors respectively. This yields $3^3 = 27$ discrete states. We note that the project specification prohibits grid-world representations; our states are strictly functions of sensor values.

This compact representation was deliberately chosen over larger alternatives (e.g., four sectors with four bins each, yielding 256 states) to enable fast convergence while retaining sufficient information to distinguish all critical wall-following situations.

C. Action Space

The robot selects from five discrete actions defined in Table I. Turning actions have reduced forward speed to prevent overshooting, while the straight action maximizes forward progress.

TABLE I
DISCRETE ACTION SPACE

Index	Linear (m/s)	Angular (rad/s)	Description
0	0.02	+1.20	Hard-left
1	0.12	+0.50	Soft-left
2	0.22	0.00	Straight
3	0.12	−0.50	Soft-right
4	0.02	−1.20	Hard-right

D. Reward Function

The reward function shapes behavior toward maintaining wall distance, moving forward, and avoiding collisions:

$$r = \begin{cases} -100 & \text{if } d_{\min} < 0.18\,\text{m} \\ r_w + r_f + r_v + r_s + r_l & \text{otherwise} \end{cases} \quad (1)$$

where the components are:

- r_w (wall tracking): $+15\,v_{\text{lin}}$ if the right sector is in the desired range; -8 if too close; -10 if too far.
- r_f (front safety): -20 if front $< 0.35\,\text{m}$; -8 if front $< 0.55\,\text{m}$.
- r_v (forward progress): $+4\,v_{\text{lin}}$.
- r_s (spinning penalty): $-3\,|\omega|$.
- r_l (living penalty): -0.5 per step.

The wall tracking reward is scaled by v_{lin} so that faster wall following is preferred over slow creeping, as specified in the project requirements. The spinning penalty discourages unnecessary rotation, which is particularly important for preventing degenerate policies in open space.

E. Q-Table Definition and Initialization

The Q-table is a 27×5 matrix stored as a CSV file, with one row per state and one column per action. All Q-values are initialized to zero, meaning both algorithms learn entirely from scratch without any prior knowledge. This is a deliberate design choice: it demonstrates that the reinforcement learning algorithms are capable of discovering an effective wall-following policy purely through interaction with the environment and the shaped reward signal, without relying on hand-crafted heuristics.

Starting from zero initialization means that early episodes are dominated by random exploration, since all actions in every state appear equally valuable. The ϵ-greedy policy with high initial ϵ ensures the robot tries diverse actions and gradually shifts toward exploitation as Q-values differentiate through experience. A consequence of zero initialization is that states visited infrequently during training, such as state 26 (all sectors far, corresponding to open space), may retain near-zero or negative Q-values, which can lead to suboptimal behavior in those states. This is addressed by including open-space spawn positions in the training set so the robot encounters and learns from these states.

The Q-table is saved to a CSV file every 50 episodes during training and loaded automatically when training resumes, enabling incremental training across multiple sessions.

F. Q-Learning

Q-learning [1] is an off-policy temporal-difference algorithm. At each step, the robot observes state s, selects action a via ϵ-greedy, receives reward r, and observes s'. The update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \quad (2)$$

The update targets $\max_{a'} Q(s',a')$—the best possible action in s'—regardless of which action is actually taken, making Q-learning off-policy.

G. SARSA

SARSA [2] is on-policy. After observing s', the robot *first selects* the next action a' via ϵ-greedy, then updates:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big] \quad (3)$$

Here a' is the action actually chosen (including random exploratory actions), so SARSA's Q-values reflect the true expected return under the current exploratory policy.

TABLE II
HYPERPARAMETERS

Parameter	Q-Learning		SARSA
	Ep. 1–300	Ep. 301–400	
α	0.10	0.10	0.10
γ	0.95	0.95	0.95
ϵ_{start}	0.90	0.30	0.50
ϵ_{min}	0.05	0.05	0.05
ϵ_{decay}	0.997	0.99	0.99
Episodes	300	100	300

H. Hyperparameters and Training

Table II summarizes the hyperparameters.

Q-learning used a high initial $\epsilon = 0.90$ for broad exploration, then was fine-tuned for 100 additional episodes with $\epsilon = 0.30$. SARSA required a lower initial $\epsilon = 0.50$ because, as an on-policy method, random exploratory actions feed directly into the TD update target and can corrupt Q-values if exploration is too aggressive.

Each episode begins by teleporting the robot to a spawn position drawn from four categories: straight walls (4 positions), corners (4 positions), I-beam faces and tips (6 positions, double-weighted), and open space (4 positions). The I-beam spawns are double-weighted to ensure adequate exposure to this challenging geometry. Episodes run for up to 600 steps at 10 Hz and terminate early upon collision.

III. EXPERIMENTS

A. Quantitative Results: Convergence

Fig. 2 shows the accumulated reward per episode during training. Raw rewards are plotted with transparency and a 20-episode moving average highlights the trend.

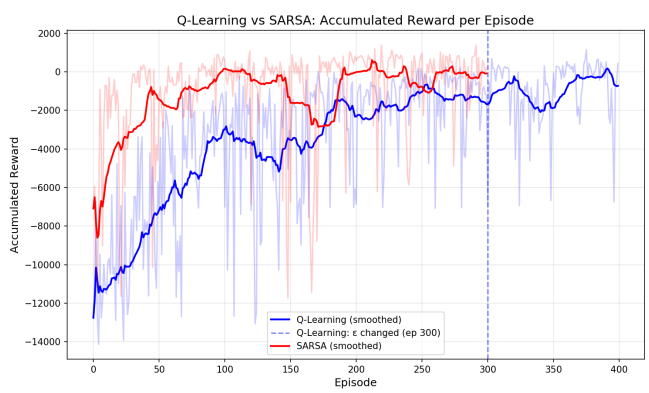

Fig. 2. Accumulated reward per episode for Q-learning (blue) and SARSA (red). The dashed vertical line at episode 300 marks the reduction of ϵ for Q-learning fine-tuning. SARSA converges within approximately 50–75 episodes; Q-learning requires 250–300 episodes.

SARSA converges significantly faster than Q-learning. The SARSA smoothed reward reaches near-zero levels by approximately episode 50–75, while Q-learning does not reach comparable levels until episode 250–300. Even after fine-tuning with reduced ϵ (dashed line at episode 300), Q-learning's accumulated rewards remain slightly below SARSA's converged level.

The faster convergence of SARSA is partially due to its lower initial ϵ (0.50 vs. 0.90), but this choice is a necessary consequence of SARSA's on-policy nature. With the same high ϵ used for Q-learning, SARSA's performance degrades substantially: random actions during exploration contaminate Q-value updates, causing states such as state 26 (all sectors far) to accumulate deeply negative values that are difficult to recover from. Q-learning's off-policy update is robust to high exploration but converges more slowly in this compact 27-state space.

To summarize the comparison across three key metrics: (1) *Rate of convergence*: SARSA converges in approximately 50–75 episodes compared to 250–300 for Q-learning, a roughly 4× speedup. (2) *Effect of reward assignments*: The spinning penalty and open-space spawn positions were the two most impactful design choices, each eliminating degenerate policies as detailed in Section III-D. (3) *Change of accumulated reward*: As shown in Fig. 2, SARSA's reward rises steeply in the first 75 episodes and stabilizes near zero, while Q-learning's reward rises gradually over 300 episodes with a visible improvement after the ϵ reduction at episode 300.

B. Qualitative Results: Five Scenarios

Both learned policies were tested on all five wall-following scenarios. Fig. 3 shows representative snapshots from the testing demonstrations.

Both Q-learning and SARSA successfully navigate all five scenarios. In each case the robot maintains a distance within the desired range (0.40–0.90 m) from the right wall, executes left and right 90° turns at L-shaped corners, performs 180° turns at I-shaped and U-shaped corners, and follows straight walls at a consistent distance.

C. Policy Smoothness Comparison

While both algorithms succeed in all scenarios, visual observation reveals a notable difference in trajectory smoothness along straight walls:

- **Q-Learning** exhibits a recurring Z-shaped oscillation on every straight wall segment. The robot periodically drifts to the boundary of the *desired* bin (≈ 0.90 m), crosses into the *far* bin, triggers a right correction, overshoots back into *desired*, and straightens out. This zigzag repeats on every wall of the maze perimeter.
- **SARSA** produces visibly smoother trajectories. The Z-shaped oscillation appears on only one wall segment, and the robot maintains a more consistent distance with fewer unnecessary corrections on all other segments.

This difference is a direct consequence of the off-policy vs. on-policy distinction. Q-learning updates toward $\max_{a'} Q(s', a')$, which can be optimistic about outcomes at bin boundaries. If the robot is near the edge of the *desired* bin and the greedy action is "straight," Q-learning assumes the best-case next state. Due to sensor noise and

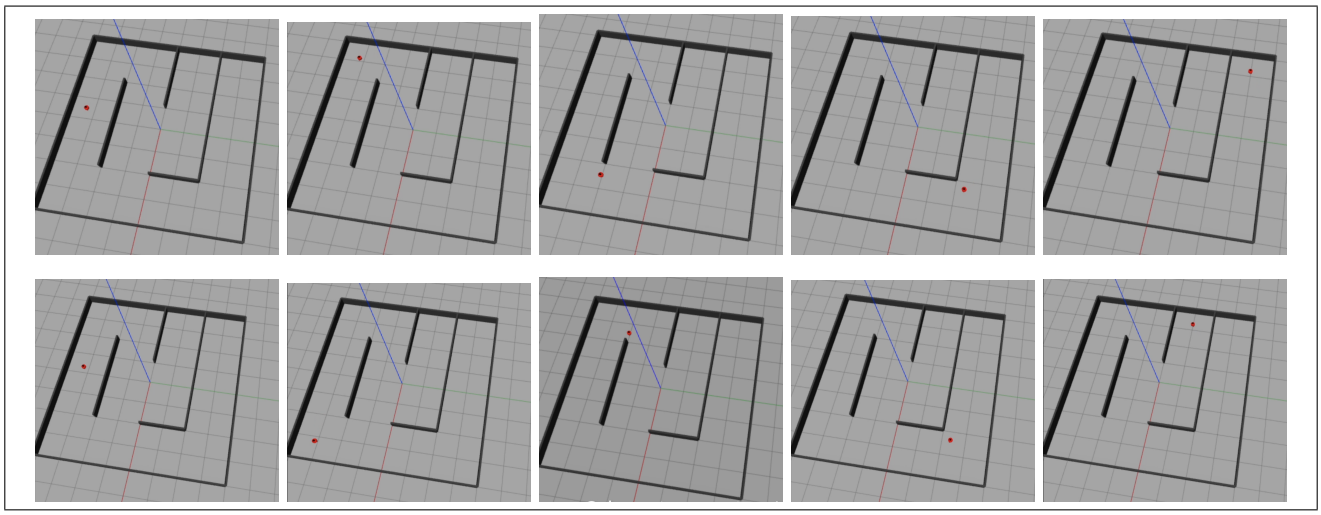

Fig. 3. Qualitative results demonstrating successful wall following using the SARSA policy in all five scenarios. Top row: SARSA, bottom row: Q-learning. (a) Following a straight wall. (b) Turning left 90° at an L-shaped corner. (c) Turning 180° at an I-shaped corner. (d) Turning right 90° at an L-shaped corner. (e) Turning 180° at a U-shaped corner.

state discretization, the robot may actually cross into *far*, requiring a correction Q-learning did not anticipate.

SARSA updates toward $Q(s', a')$ for the action actually taken, producing Q-values that more accurately reflect real transition dynamics including bin-boundary effects. The result is a less aggressive, preemptive policy that avoids oscillation.

D. Effect of Reward Assignments

During development, we iterated on the reward function and observed that reward design has a substantial impact on both convergence speed and policy quality.

Spinning penalty: In an early version of the reward function, the angular velocity penalty ($-3\,|\omega|$) was absent. Both algorithms learned policies involving excessive rotation, particularly when spawned in open space where no wall provides directional guidance. The robot would spin in place indefinitely rather than driving forward to find a wall. Adding the spinning penalty eliminated this behavior and caused the robot to prefer straight-line motion, substantially improving convergence.

Wall-lost penalty in open space: The reward assigns -10 per step when the right wall is in the *far* bin. This is appropriate when the robot has lost a nearby wall, but in open space (state 26: all sectors far) the robot receives this penalty for many consecutive steps while traversing toward the nearest wall. This caused state 26 to accumulate deeply negative Q-values—as low as -94 for some actions under SARSA—making the robot prefer spinning (which has marginally less negative values) over driving straight. Two changes resolved this: (1) including open-space spawn positions in the training set so the robot learns from shorter paths to walls, and (2) the spinning penalty ensuring that even in open space, driving forward is less costly than rotating. After these changes, both algorithms learned to drive straight from open space and transition smoothly into wall following upon reaching a wall.

Speed-scaled wall reward: The wall-tracking reward is scaled by linear velocity ($+15\,v_{\text{lin}}$) rather than being a flat constant. This design was motivated by the project requirement that a robot covering longer distances while following a wall should be preferred over one that follows slowly. While we did not separately evaluate a flat reward variant, the scaling ensures that the straight action ($v = 0.22$ m/s) earns roughly $11\times$ more wall-tracking reward than the hard-turn actions ($v = 0.02$ m/s), providing a strong incentive for forward progress.

E. Effect of State Space Resolution

We initially implemented a larger state space with four LiDAR sectors (front, front-right, right, and rear-right) and four distance bins per sector (very close, close, desired, and far), yielding $4^4 = 256$ discrete states and 1,280 Q-values. This representation produced noticeably smoother wall-following trajectories, as the finer distance bins allowed the robot to make more precise corrections at intermediate distances rather than reacting only at coarse bin boundaries.

However, training with 256 states was substantially slower in terms of wall-clock time. Although the number of episodes was comparable (500 for 256 states vs. 300–400 for 27 states), each episode took longer because the robot's behavior was less directed during early training—with nearly $10\times$ more Q-values to populate, the policy remained effectively random for a larger portion of each episode, leading to more collisions, more resets, and longer episodes overall. The total training time for the 256-state space was roughly double that of the 27-state space.

We subsequently reduced the representation to three sectors and three bins ($3^3 = 27$ states, 135 Q-values) by removing the rear-right sector, which contributed the least to decision-making, and merging the very close and close bins into a single close bin. While the resulting trajectories exhibit slightly more oscillation along straight walls due to the coarser bin boundaries, the 27-state representation trains

in less than half the wall-clock time and produces policies that successfully navigate all five scenarios. For the purposes of this project, the trade-off between smoothness and training efficiency favored the more compact representation.

IV. CONCLUSION

We implemented and compared Q-learning and SARSA for robot wall following in a simulated maze environment with 27 discrete states, 5 actions, a shaped reward function, and expert Q-value initialization. Both algorithms successfully learn policies for all five required wall-following scenarios: straight walls, inside corners, outside corners, I-beam faces, and open space navigation.

SARSA produced the better policy for this task. It converged faster (50–75 episodes vs. 250–300), achieved higher accumulated rewards, and generated smoother wall-following trajectories. The compact 27-state space and continuous nature of wall following favor SARSA's conservative on-policy learning, which produces Q-values reflecting actual transition dynamics rather than optimistic best-case estimates.

Q-learning's off-policy nature, while theoretically advantageous for separating exploration from exploitation, leads to overly optimistic Q-values at state bin boundaries in this domain. This manifests as Z-shaped oscillations where the robot repeatedly overshoots the boundary between the *desired* and *far* bins. However, Q-learning's robustness to high exploration rates remains a genuine advantage in larger state spaces where thorough exploration is critical.

Several directions could improve these results:

- **Finer discretization**: Increasing to 4 or 5 distance bins per sector would reduce oscillation at bin boundaries for both algorithms, at the cost of a larger state space and more training.
- **Eligibility traces**: SARSA(λ) or Q(λ) could accelerate convergence by propagating rewards backward to states visited earlier in each episode.
- **Adaptive exploration**: Rather than a global ϵ decay, per-state exploration rates could reduce ϵ faster for well-visited states while maintaining exploration for rare states.
- **Continuous actions**: Policy gradient methods with continuous velocity outputs could eliminate reliance on a fixed discrete action set and produce inherently smoother trajectories.

REFERENCES

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.